JAVA ASSIGNMENT

Exception Handling & Input / Output

A balanced assessment of concepts, control flow, stream selection and robust file processing.

05
questions

03 THEORY

02 PRACTICAL

50 TOTAL MARKS

Each question carries 10 marks

Answer all questions • Show code and reasoning where requested

QUESTION 1

THEORY

Explain how Java handles exceptional situations

Explain Java’s exception hierarchy and distinguish errors, checked exceptions, and unchecked exceptions.

Then describe how try, catch, finally, throw and throws work together. Support your answer with one short Java example.

Your answer should make the decision points clear: what the compiler enforces, what happens at runtime, and who is responsible for recovery.

MARK BREAKDOWN

Hierarchy & concepts	3
Checked vs unchecked	3
Keywords + example	4

10

marks

Trace exception control flow and propagation

Study the program. State its exact output sequence, then explain why each line does—or does not—execute.

```
static void read() throws IOException {
  throw new IOException("data");
}

public static void main(String[] args) {
  try { read(); System.out.println("A"); }
  catch (IOException e) {
System.out.println("B"); }
  finally { System.out.println("C"); }
}
```

01 OUTPUT

Write the lines in their exact order.

02 PROPAGATION

Explain how IOException reaches the catch block.

03 FINALLY

Explain the role of finally in this execution.

MARKS 3 output • 4 reasoning • 3 finally

Select the right stream for the data

Compare Java byte streams and character streams. Your answer must connect the abstraction to the kind of data being processed.

BYTE STREAMS

InputStream / OutputStream

- Raw binary data
- FileInputStream / FileOutputStream
- Appropriate for images, PDFs and archives

CHARACTER STREAMS

Reader / Writer

- Text decoded as characters
- FileReader / FileWriter
- Often wrapped with BufferedReader / BufferedWriter

DECISION TASK

Recommend an API for (a) reading a UTF-8 configuration file and (b) copying a JPEG. Justify both choices and explain what buffering changes.

4 comparison • 4 selection & reasoning • 2 buffering

Build a resilient student-record file reader

Write a Java program that reads students.txt, where each line follows name,score.

Sample input

```
Asha,78
Ravi,abc
Meera,35
Dev,105
```

Use BufferedReader and BufferedWriter. Do not stop processing when one line is invalid.

REQUIREMENTS

01 Validate the format and ensure score is 0–100.

02 Write records with score ≥ 40 to passed.txt.

03 Write malformed or out-of-range lines to errors.txt.

04 Handle file errors and close every resource safely.

4 logic • 3 exceptions & resources • 2 output/logging • 1 clarity

Copy a binary file safely and efficiently

Create a command-line Java program that copies any binary file from a source path to a destination path.

01

READ PATHS

Accept source and destination paths from args[].

02

COPY IN CHUNKS

Use buffered byte streams and a fixed-size byte array.

03

REPORT & RECOVER

Print bytes copied and handle missing source or write failure separately.

ACCEPTANCE CHECK

- ✓ Uses InputStream / OutputStream
- ✓ Wraps streams for buffering
- ✓ Uses try-with-resources
- ✓ Never loads the full file into memory
- ✓ Produces a byte-for-byte copy

MARKS

4 copy logic • 2 resources
2 error handling • 2 CLI & reporting